A Mobile Phone based Voice-Controlled UGV

Divyansh Maheshwari, Shriyansh Chawda and GVV Sharma
Department of Electrical Engineering
Indian Institute of Technology, Hyderabad
Kandi, India 502284
gadepall@ee.iith.ac.in

***Abstract*—This paper implements a cloud-free voice-controlled Unmanned Ground Vehicle (UGV), consisting of a simple speech recognition engine and microcontroller based mobility. The novelty of this work lies in the fact that all voice commands are processed on an Android smartphone where all speech recognition algorithms are implemented. The key algorithms used are Mel-Frequency Cepstral Coefficient (MFCC) features combined with Dynamic Time Warping (DTW). User adaptation is supported through up to ten recorded templates per command, achieving 90–95% accuracy across 10 speakers and demonstrating language-agnostic operation in English, Hindi, Gujarati, and Tamil.**

***Index Terms*—UGV, voice control, MFCC, DTW, ESP32, BLE, Android, speech recognition, embedded systems**

I. Introduction

Speech-based control removes the need for physical controllers and lowers the barrier for non-expert operation of ground robots. Most prior implementations use recognition to cloud ASR services [1], which introduces round-trip latency and ties the system to network availability. Running a full speech model on the robot microcontroller directly is equally impractical: a device such as the ESP32 carries insufficient flash and RAM for any modern recogniser.

This paper resolves the compute–connectivity dilemma by offloading all recognition to the Android smartphone that the operator already carries. Short utterances are converted to MFCC feature matrices [2] and matched against stored templates using DTW [3]. Both steps run entirely on the phone, so no audio ever leaves the device. Neural keyword-spotting alternatives exist for microcontrollers [4], [5] but demand labelled training corpora and quantisation pipelines that a DTW system avoids entirely. A 7-byte BLE command string is transmitted to the ESP32 only after a match is confirmed, reducing the wireless payload by over 99% compared to streaming raw audio [6].

The specific contributions of this work are:

1) A smartphone-resident MFCC+DTW pipeline that operates without any cloud or internet dependency.
2) A Best-of-N template strategy allowing each user to enrol up to ten personal utterances per command for speaker adaptation.
3) A latency-optimised BLE scheme that cuts end-to-end response time from 2–4 s to approximately 1.6 s by transmitting only the recognised command label.

II. System Architecture

A. Overview

Two hardware nodes form the complete system: an Android smartphone that handles all audio capture and recognition, and an ESP32 microcontroller that handles BLE reception and motor actuation.

An earlier design ran MFCC extraction and DTW matching on the ESP32 itself. That approach consumed ∼80 kB of flash, streamed 24,000 bytes of raw audio over BLE per utterance, and produced 2–4 s of end-to-end latency. Moving recognition to the phone shrinks the BLE payload to a 7-byte ASCII command string and cuts latency to ∼1.6 s. The ESP32 is left with only two tasks: receive the string and drive the motors.

TABLE I
Architecture Comparison: Old vs. New Pipeline

Stage	Old	New
Audio capture	ESP32 mic	Android mic
MFCC extraction	ESP32 (C++)	Android (Java)
DTW matching	ESP32 (C++)	Android (Java)
BLE payload	24,000 bytes	7 bytes
ESP32 flash	∼80 kB extra	Minimal
End-to-end latency	2–4 s	∼1.6 s

B. Data Flow

The pipeline executes five steps on each button press:

1) The Android app records 1.5 s of 8 kHz mono audio (12,000 samples, 16-bit PCM).
2) `MFCCProcessor` produces a 92×26 feature matrix — 13 static and 13 delta coefficients per frame.
3) `DTWMatcher` scores the query against every stored template for all five commands using Best-of-N selection.
4) If the lowest score falls below $\tau = 12.0$, the corresponding command string is written to the BLE characteristic.
5) The ESP32 reads the string and actuates the L293D motor driver.

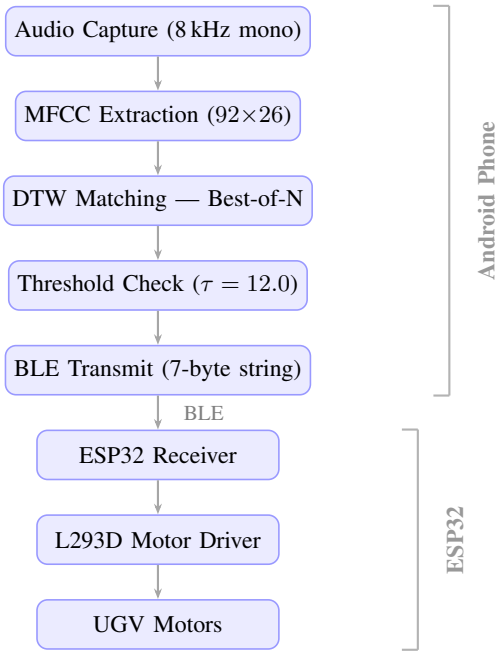

Fig. 1. System pipeline block diagram.

III. Hardware Setup

A. Components

TABLE II
List of Components

Component	Qty.	Description
Chassis frame	1	Holds all components
Side wheels (DC motors)	2	Drive wheels
Caster wheel	1	Stability and smooth turning
ESP32 dev board	1	Microcontroller + BLE
L293D motor driver IC	1	Drives DC motors
DC motors (TT gear)	2	Propulsion
Breadboard	1	Circuit connections
Jumper wires	11	Electrical connections
Power bank / LiPo	1	Portable power supply
Android phone (API 23+)	1	Recognition + BLE central
Micro-USB / USB-C cable	1	ESP32 flashing

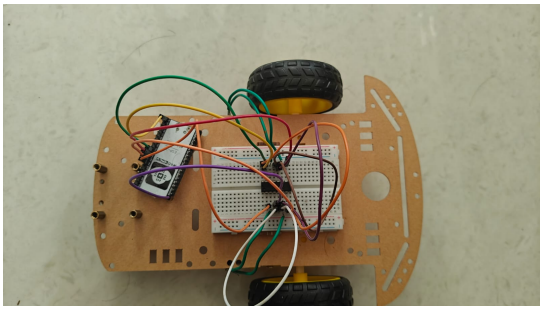

Fig. 2. Top-down view of the assembled UGV prototype, detailing the ESP32, breadboard, L293D motor driver, and chassis wiring.

B. Motor Wiring

Four GPIO pins on the ESP32 drive the L293D input lines directly using digital writes — no PWM is involved. EN1 and EN2 are tied permanently HIGH so both motor channels remain enabled at all times.

TABLE III
ESP32 to L293D Connections

ESP32 GPIO	L293D Pin	Function
GPIO 32	IN1	Motor A — direction 1
GPIO 33	IN2	Motor A — direction 2
GPIO 25	IN3	Motor B — direction 1
GPIO 26	IN4	Motor B — direction 2
GND	GND	Common ground
3.3 V	EN1, EN2	Motor channel enable (HIGH)

C. Full Schematic

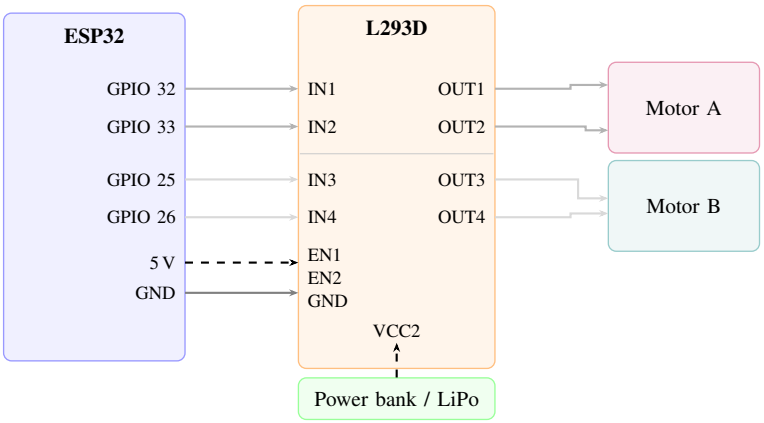

Fig. 3. Wiring schematic: ESP32 to L293D to DC motors. Dark arrows — Motor A channel (GPIO 32/33 → IN1/IN2 → OUT1/OUT2). Light arrows — Motor B channel (GPIO 25/26 → IN3/IN4 → OUT3/OUT4). Dashed — power connections (3.3 V to EN1, EN2; motor supply to VCC2).

D. Motor Truth Table

TABLE IV
Motor Direction Truth Table

IN1	IN2	IN3	IN4	Result
H	L	H	L	Move Forward
L	H	L	H	Move Back
L	H	H	L	Turn Left
H	L	L	H	Turn Right
L	L	L	L	Motor Stop

IV. Signal Processing — MFCC Feature Extraction

Each 1.5 s recording is converted to a 92×26 feature matrix through seven sequential stages. Full equations are in Appendix A.

A. Framing Parameters

The 12,000-sample recording is cut into overlapping frames of $N = 256$ samples (32 ms) with a hop of $H = 128$ samples (16 ms), yielding $(12000 - 256)/128 + 1 = 92$ frames.

B. Pre-emphasis

A first-order filter with $\alpha = 0.97$ (Eq. 1) attenuates low-frequency dominance. Samples are scaled to $[-1, 1]$ by dividing by 32768 before windowing.

C. Hamming Window

A Hamming window (Eq. 2) is applied per frame to suppress spectral leakage at frame boundaries.

D. Fast Fourier Transform and Power Spectrum

A 256-point radix-2 FFT is computed per frame. The one-sided power spectrum (Eq. 3) retains $N/2 + 1 = 129$ bins.

E. Mel Filterbank

Twenty-six triangular filters, uniformly spaced on the Mel scale (Eq. 4), are applied to the power spectrum. Log energy is summed per filter (Eq. 5); a floor of 10^{-10} prevents $\log(0)$.

F. Discrete Cosine Transform

DCT-II with orthonormal weights (Eqs. 6–7) decorrelates the 26 log-energies into 13 cepstral coefficients.

G. Delta Coefficients

First-order temporal differences (Eq. 8) are appended to each frame, expanding the feature vector from 13 to 26 dimensions. Boundary frames replicate the nearest valid neighbor.

H. Cepstral Mean and Variance Normalisation (CMVN)

CMVN (Eqs. 9–11) standardises each coefficient across the 92 frames, with $\varepsilon = 10^{-8}$ preventing division by zero. The output is a 92×26 matrix passed directly to the DTW matcher.

V. Pattern Matching — Dynamic Time Warping

A. DTW Algorithm

A query matrix $\mathbf{Q} \in \mathbb{R}^{F_1 \times 26}$ is aligned against a template $\mathbf{T} \in \mathbb{R}^{F_2 \times 26}$ using Euclidean frame distance (Eq. 12). The accumulated cost matrix D is filled left-to-right via the three-way recurrence in Eq. 13. A parallel matrix L (Eq. 14) counts steps along the warping path so that the final score (Eq. 15) is normalised by path length, removing the duration bias that arises when comparing utterances of unequal length. Storing only two rolling rows of D and L reduces memory from $O(F_1 F_2)$ to $O(F_2)$.

B. Best-of-N Strategy

Each command holds up to $N = 10$ templates: index 0 is the original bundled template, indices 1–9 are recorded by the user in-app. The score assigned to a command is the minimum across all its templates (Eq. 16). The command with the lowest score is accepted only if that score falls below $\tau = 12.0$ (Eq. 17). Table VI lists the score ranges observed in practice.

VI. Recognition Algorithm

This section presents the complete end-to-end voice recognition pipeline as formal algorithms. Algorithm 1 details MFCC extraction and Algorithm 2 details DTW recognition with Best-of-N template matching.

Algorithm 1 MFCC Feature Extraction

Require: Raw audio samples $x[n]$, $n = 0, \ldots, 11999$ (1.5 s at 8 kHz)
Ensure: Feature matrix $\hat{\mathbf{C}} \in \mathbb{R}^{92 \times 26}$

```
Pre-emphasis: y[n] ← x[n] − 0.97 x[n−1]; normalize by 32768
for each frame f = 0 to 91 do
    Extract samples y[fH : fH + N] with N = 256, H = 128
    Apply Hamming window: s[n] ← y[n] · w[n]
    Compute 256-point FFT; form power spectrum P[k] = |X[k]|^2/N
    Apply 26-band Mel filterbank; compute log energies E[m]
    Apply DCT-II to E[m]; retain first 13 coefficients c[f]
end for
for each frame f = 0 to 91 do
    Compute delta: Δc[f][k] ← (c[f+1][k] − c[f−1][k])/2
    Concatenate: c[f] ← [c[f] ‖ Δc[f]] (26-dim)
end for
Compute per-coefficient mean μ_k and std σ_k across all frames
CMVN: ĉ[f][k] ← (c[f][k] − μ_k)/(σ_k + 10^−8)
return Ĉ
```

Algorithm 2 Voice Command Recognition via DTW + Best-of-N

Require: Query MFCC matrix $\mathbf{Q} \in \mathbb{R}^{F_1 \times 26}$, template sets $\{\mathbf{T}_{c,i}\}$ for each command c, threshold $\tau = 12.0$
Ensure: Recognized command $\hat{c}$, or REJECT

for each command $c \in \{$*forward, back, left, right, stop*$\}$ **do**
  $s_c \leftarrow \infty$
  for each stored template $\mathbf{T}_{c,i}$ of length F_2 **do**
    Initialize $D_{\text{prev}}[j] \leftarrow \infty$, $L_{\text{prev}}[j] \leftarrow 0$ for $j = 0, \ldots, F_2{-}1$
    for $i = 0$ **to** $F_1 - 1$ **do**
      for $j = 0$ **to** $F_2 - 1$ **do**
        $d \leftarrow \|\mathbf{Q}[i] - \mathbf{T}_{c,i}[j]\|_2$
        Let $a = D[i{-}1][j]$, $b = D[i][j{-}1]$, $c = D[i{-}1][j{-}1]$; $p^* \leftarrow \arg\min(a, b, c)$
        $D_{\text{curr}}[j] \leftarrow d + D_{p^*}$; $L_{\text{curr}}[j] \leftarrow 1 + L_{p^*}$
      end for
      $D_{\text{prev}} \leftarrow D_{\text{curr}}$; $L_{\text{prev}} \leftarrow L_{\text{curr}}$
    end for
    score $\leftarrow D_{\text{prev}}[F_2{-}1] \,/\, L_{\text{prev}}[F_2{-}1]$
    $s_c \leftarrow \min(s_c,\ \text{score})$ {Best-of-N}
  end for
end for
$\hat{c} \leftarrow \arg\min_c\ s_c$
if $s_{\hat{c}} < \tau$ **then**
  return $\hat{c}$
else
  return REJECT
end if

VII. Software Implementation

A. ESP32 Firmware

Three source files make up the firmware, built with PlatformIO on the Arduino framework.

ble_receive.cpp sets up a BLE GATT server with a WRITE characteristic (`UUID: ...9002`) for incoming command strings and a NOTIFY characteristic (`UUID: ...9003`) for acknowledgements. Any string not matching the five known commands is discarded. Advertising for new connection restarts automatically on disconnect.

motor.cpp toggles four GPIO pins (32, 33, 25, 26) to drive the L293D. No PWM is used.

main.cpp implements the command dispatcher:

Listing 1. ESP32 command dispatcher (main.cpp)

```
void loop() {
  char cmd[16] = {0};
  if (!ble_receive_command(cmd, sizeof(cmd)))
    { delay(5); return; }

  if (!strcmp(cmd,"forward")) { motor_forward(); }
  else if (!strcmp(cmd,"back")) { motor_back(); }
  else if (!strcmp(cmd,"left"))
    { motor_left(); delay(600); motor_stop(); }
  else if (!strcmp(cmd,"right"))
    { motor_right(); delay(600); motor_stop(); }
  else if (!strcmp(cmd,"stop")) { motor_stop(); }}
```

forward and *back* run until a *stop* is received. *left* and *right* run for 600 ms then auto-stop, giving approximately a 90° turn. The duration is adjustable for different chassis.

B. Android Application

The app records 1.5 s at 8 kHz, runs MFCC extraction and DTW matching on a background thread, and posts results to the UI thread. A score below τ triggers a BLE write of the command string. A debug screen displays per-command DTW scores live; a training screen lets users record additional templates. A 200 ms pause before each `AudioRecord` initialisation prevents microphone release errors on back-to-back recordings.

C. Template Conversion Script

`convert_templates.py` is a one-time script that reads the C header `templates.h`, strips the `f` suffix from float literals, and writes one `.bin` file per command for `TemplateStore` using `struct.pack`.

D. Out-of-the-Box Deployment

Flashing the firmware and installing the APK is all that is required. The app scans for a BLE device named `ESP32_UGV` and connects automatically. Once paired, the user speaks a command and the vehicle responds within approximately 1.6 s.

VIII. User Adaptation and Tuning

A. Recording Additional Templates

The `TemplateStore` always holds template index 0, which ships pre-built inside `assets` and was recorded by the original developer. Slots 1–9 are filled on the target device through `DebugActivity`'s record button; no PC or cable is needed after the initial APK install. How many slots to fill depends on the use case — Table V gives concrete guidance.

TABLE V
Recommended Template Count per Scenario

Scenario	Min.	Recommended
Quick prototype, quiet room	1	3–5
Daily driver, single speaker	5	8–10
Noisy environment	8	10
Shared templates across speakers	Not supported	

B. Threshold Tuning

Picking a good τ is straightforward with the live score display in the debug screen. Run several genuine utterances and note the highest DTW score that still matches the correct command; then run a few wrong commands and note the lowest score that correctly rejects them. Set τ anywhere in the gap between those two numbers. In practice, 8–10 personal templates recorded in a quiet room push the correct-command scores down to the 3.0–7.5 range (Table VI), so a threshold of 8.0–10.0 gives comfortable separation for a single speaker.

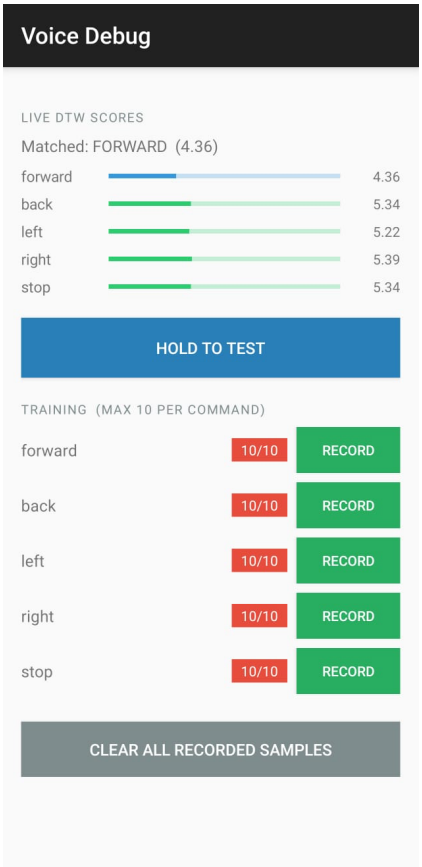

Fig. 4. Debug Activity screen showing live DTW scores after speaking *forward*. The blue bar indicates the winning command (score 4.36), while all other commands score above 5.0. All five commands have 10 recorded templates (10/10), producing well-separated scores and reliable recognition.

C. Turn Duration

Lateral movement is timed rather than angle-sensed: `main.cpp` holds the turn GPIO pattern for 600 ms before issuing a stop, which yields roughly 90° on the development chassis at full motor voltage. Heavier or slower builds need a longer delay; faster or lighter ones need a shorter one. The constant is a single integer literal in `main.cpp` and requires no recompilation of any Android code.

IX. Results and Discussion

A. Multi-Speaker Evaluation

Ten volunteers each recorded 10 templates per command through the in-app training interface, then independently spoke each command multiple times in a quiet indoor room. Across all speakers and all five commands the system maintained **90–95%** accuracy, as broken down in Table VI.

TABLE VI
Typical DTW Scores (single speaker, quiet environment)

Condition	Score Range	Outcome
Correct command, 10 templates	3.0 – 7.5	Match ($< \tau$)
Correct command, 1 template	5.0 – 11.0	Match / borderline
Wrong command	11.0 – 22.0	No match ($> \tau$)
Silent / noise	18.0 – 30.0	No match

DTW is fundamentally a speaker-dependent algorithm, so achieving this accuracy across 10 different voices warrants explanation. Two mechanisms are responsible. First, CMVN (Equation 11) subtracts the per-coefficient mean and divides by the standard deviation computed over the 92-frame window, stripping out slow-varying spectral biases that differ between speakers. Second, the Best-of-N rule (Equation 16) picks the single closest template out of up to 10 stored ones, so each user's personal recordings effectively anchor the matcher to their own vocal characteristics.

B. Cross-Language Evaluation

A subset of participants issued commands in **Hindi, Gujarati, and Tamil** rather than English. Accuracy stayed within the 90–95% band across all three languages. No changes to the pipeline were needed: because `MFCCProcessor` works on raw acoustic energy and `DTWMatcher` compares 26-dimensional frame vectors, the system carries no phoneme tables, pronunciation dictionaries, or language models that would tie it to a specific language. Any consistently pronounced word maps to a repeatable MFCC trajectory and can therefore serve as a command.

C. Latency

Button press to motor response measured at roughly 1.6 s. The previous ESP32-resident pipeline took 2–4 s because it had to receive all 24,000 bytes of 8 kHz PCM over BLE before extraction could begin. Shifting MFCC and DTW to the phone reduces the BLE transfer to a 7-byte ASCII string, and that single change accounts for most of the latency improvement.

D. Limitations

Each new speaker must record their own templates; there is currently no speaker-independent enrolment path. In noisy settings the `NoiseSuppressor` API offers some relief but does not fully recover accuracy. On the ESP32 side, the 600 ms fixed turn timer is a heuristic — it produces approximately 90° on the test chassis but needs manual adjustment for any build with different motor speed or chassis weight. Addressing all three points — speaker independence, noise robustness, and closed-loop turning — is left to future work.

X. Conclusions

Relocating MFCC extraction and DTW matching from the ESP32 to an Android smartphone produces a fully on-device, cloud-free voice control system for a UGV. The shift cuts end-to-end latency from 2–4 s to 1.6 s and recovers 80 kB of microcontroller flash. CMVN normalisation and the Best-of-N

template strategy together yield 90–95% accuracy across 10 speakers and four languages — English, Hindi, Gujarati, and Tamil — despite DTW being inherently speaker-dependent.

Two limitations remain. Each new speaker must record personal templates, and accuracy drops in noisy environments even with Android's `NoiseSuppressor` API active. Future work will investigate compact convolutional or recurrent keyword-spotting models that run on-device without per-user enrolment, alongside data augmentation for noise robustness and a larger-scale speaker evaluation.

References

[1] G. Li, S. Mei, H. Yang, X. Zhang, H. Wang, and F. Sun, "Speech interaction with robots: A review," *International Journal of Advanced Robotic Systems*, vol. 14, no. 6, 2017.

[2] S. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 28, no. 4, pp. 357–366, 1980.

[3] H. Sakoe and S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 26, no. 1, pp. 43–49, 1978.

[4] Y. Zhang, N. Suda, L. Lai, and V. Chandra, "Hello edge: Keyword spotting on microcontrollers," in *arXiv preprint arXiv:1711.07128*, 2017.

[5] S. Choi, S. Seo, and B. Moon, "An efficient keyword spotting technique using a recurrent neural network for resource-constrained devices," *Applied Sciences*, vol. 9, no. 9, p. 1885, 2019.

[6] K. Mikhaylov and J. Tervonen, "Evaluation of power consumption for Bluetooth Low Energy," in *2013 IEEE 24th Annual International Symposium on Personal, Indoor, and Mobile Radio Communications (PIMRC)*. IEEE, 2013, pp. 155–159.

[7] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," *arXiv preprint arXiv:1804.03209*, 2018.

[8] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[9] A. Maier *et al.*, "Design and implementation of ESP32-based IoT devices," *Sensors*, vol. 23, no. 15, p. 7023, 2023.

[10] J. Gonzalez *et al.*, "Implementation of a speech recognition system in a DSC," *IEEE Latin America Transactions*, vol. 14, no. 6, pp. 2657–2662, 2016.

Appendix

This appendix collects all equations referenced in Sections IV and V.

B.1 MFCC Feature Extraction

Pre-emphasis:

$$y[n] = x[n] - \alpha\, x[n-1], \quad \alpha = 0.97 \tag{1}$$

Hamming window:

$$w[n] = 0.54 - 0.46\cos\left(\frac{2\pi n}{N-1}\right), \quad n = 0, \ldots, N-1 \tag{2}$$

Power spectrum:

$$P[k] = \frac{|X[k]|^2}{N}, \quad k = 0, \ldots, \frac{N}{2} \tag{3}$$

Mel scale:

$$m = 2595 \log_{10}\left(1 + \frac{f}{700}\right) \tag{4}$$

Mel filterbank log energy:

$$E[m] = \log\left(\sum_k P[k]\, H_m[k]\right), \quad m = 0, \ldots, 25 \tag{5}$$

DCT-II coefficients:

$$c[k] = w_k \sum_{n=0}^{M-1} E[n] \cos\left(\frac{\pi k(2n+1)}{2M}\right), \quad k = 0, \ldots, 12 \tag{6}$$

$$w_k = \begin{cases} \sqrt{1/M} & k = 0 \\ \sqrt{2/M} & k > 0 \end{cases} \tag{7}$$

where $M = 26$.

Delta coefficients:

$$\Delta c[f][k] = \frac{c[f{+}1][k] - c[f{-}1][k]}{2} \tag{8}$$

CMVN:

$$\mu_k = \frac{1}{F} \sum_{f=1}^{F} c[f][k] \tag{9}$$

$$\sigma_k = \sqrt{\frac{1}{F} \sum_{f=1}^{F} \big(c[f][k] - \mu_k\big)^2} \tag{10}$$

$$\hat{c}[f][k] = \frac{c[f][k] - \mu_k}{\sigma_k + \varepsilon} \tag{11}$$

where $\varepsilon = 10^{-8}$.

B.2 Dynamic Time Warping

Local distance:

$$d(i,j) = \|\mathbf{Q}[i] - \mathbf{T}[j]\|_2 \tag{12}$$

Accumulated cost recurrence:

$$D[i][j] = d(i,j) + \min\big(D[i{-}1][j],\ D[i][j{-}1],\ D[i{-}1][j{-}1]\big) \tag{13}$$

Path-length tracking:

$$L[i][j] = 1 + L[p^*] \tag{14}$$

where $p^* = \arg\min\big(D[i{-}1][j],\ D[i][j{-}1],\ D[i{-}1][j{-}1]\big)$.

Normalized DTW score:

$$\text{score} = \frac{D[F_1{-}1][F_2{-}1]}{L[F_1{-}1][F_2{-}1]} \tag{15}$$

Best-of-N command score:

$$s(\text{cmd}) = \min_{i=0}^{N-1} \text{score}(\mathbf{Q},\, \mathbf{T}_i) \tag{16}$$

Decision rule:

$$\hat{c} = \arg\min_{\text{cmd}}\ s(\text{cmd}), \quad \text{if } \min_{\text{cmd}} s(\text{cmd}) < \tau \tag{17}$$

Appendix

This appendix provides step-by-step instructions for setting up the complete development environment, flashing the ESP32 firmware, and building and installing the Android application.

A.1 Prerequisites

The following tools must be installed on the host PC before proceeding:

- **Python 3.8+** — required for template conversion and PlatformIO's build system. Available at https://www.python.org.
- **Git** — for cloning the repository. Available at https://github.com/gadepall/voice-ugv.git.
- **VS Code** (recommended) or any editor with PlatformIO support.

A.2 ESP32 Firmware

1) **Install PlatformIO.** Install the PlatformIO IDE extension in VS Code, or install the PlatformIO Core CLI via:

```
pip install platformio
```

2) **Clone / extract the project.** Place the `ugv_esp32/` folder on your PC.
3) **Connect the ESP32** to the PC via USB.
4) **Build and flash** the firmware:

```
pio run --target upload
```

PlatformIO automatically installs the `ESP32 BLE Arduino` library declared in `platformio.ini`. After flashing, the ESP32 advertises itself as `ESP32_UGV` over BLE and awaits a connection.

5) **Verify** by opening the serial monitor (`pio device monitor`, 115200 baud) and confirming the message `BLE started, advertising...`.

A.3 Android Application

Option A — Install the pre-built APK (recommended):

1) Copy `ugv_android.apk` to the Android device.
2) On the device, enable *Settings* → *Security* → *Install unknown apps* for your file manager.
3) Open the APK file and tap *Install*.

Option B — Build from source with Android Studio:

1) **Install Android Studio** (Hedgehog or later) from https://developer.android.com/studio. During setup, install the Android SDK (API level 33+) and the Android NDK if prompted.
2) **Open the project.** In Android Studio, choose *File* → *Open* and select the `ugv_android/` folder. Wait for Gradle sync to complete.
3) **Build the APK.** Select *Build* → *Build Bundle(s) / APK(s)* → *Build APK(s)*. The signed debug APK is placed in:

```
app/build/outputs/apk/debug/app-debug.apk
```

4) **Deploy to device.** Connect an Android phone (USB debugging enabled), then click *Run* → *Run 'app'* or use:

```
adb install app/build/outputs/apk/debug/app-
    debug.apk
```

A.4 First-Time Setup on the Phone

1) Enable Bluetooth on the Android device.
2) Open the **UGV Controller** app. The *SetupActivity* wizard launches automatically on first run, guiding you through BLE pairing.
3) Tap *Connect to ESP32*. The app scans for a BLE device named `ESP32_UGV` and connects automatically.
4) Follow the on-screen prompts to record 5 templates per command (10 recommended for best accuracy).
5) Tap any command button to issue voice commands. The recognized command and DTW score are shown on the main screen; detailed per-command scores are available in the *Debug* screen.

A.5 Troubleshooting

- **AudioRecord failed:** A 200 ms delay is already applied between recordings. If the error persists, close other apps using the microphone.
- **BLE not connecting:** Ensure the ESP32 is powered and advertising (`ESP32_UGV` appears in a BLE scanner app). Force-stop the UGV app and reconnect.
- **Poor recognition accuracy:** Record additional templates in a quiet environment. Reduce the threshold τ in `DTWMatcher.java` if valid commands are being rejected.
- **Gradle build errors:** Ensure JDK 17 is selected in *File* → *Project Structure* → *SDK Location*.